

Algunos EJEMPLOS FINALES



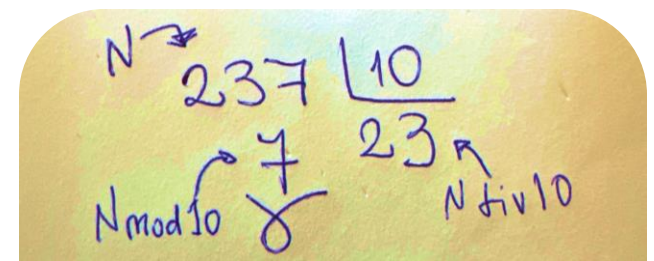
Problema Número Loco: Decimos que un número es loco si no hay ningún par de dígitos que estén adyacentes (uno al lado del otro) y sean iguales.

Por ejemplo: 235, 787, 912519 son locos.

En cambio los siguientes números no son locos: 5441 (se repite el 4 en posiciones adyacentes, la de las decenas y la de las centenas), 14211 (se repite el 1 en posiciones adyacentes, la de las unidades y la de las decenas).

Escribir un programa que reciba un número de al menos dos dígitos (validar) y determine si es o no un número loco.

Podemos asumir que cualquier número de un solo dígito es loco, ya que no tiene dos dígitos iguales adyacentes (directamente no tiene dos dígitos).



Problema Mercadito: Se mantiene una lista de información sobre productos de un mercado familiar. Esta lista esta formada por ternas de números enteros positivos, donde cada elemento representa CODIGO DE PRODUCTO, PRECIO DE PRODUCTO y CANTIDAD EN STOCK respectivamente.

Ejemplo de lista vista como números en secuencia:

12 450 35 5 1215 3 8 280 40 30 930 0 27 830 12 43 340 0 21 560 2

Se desea escribir un programa que, dada la cantidad de productos (en este caso sería 7 ya q tenemos información para 7 productos), procese una lista como esta, y muestre por pantalla:

- * Los códigos de todos los productos de los cuales no queda stock.
- * La cantidad de productos sin stock.
- * El código y precio del producto más caro.



TERNA: CODIGO DE PRODUCTO PRECIO DE PRODUCTO STOCK

12 450 35 5 1215 3 8 280 40 30 930 0 27 830 12 43 340 0 21 560 2

Recibo inicialmente la cantidad de productos (cantidad de ternas).

Mostrar por pantalla:

- * Los códigos de todos los productos de los cuales no queda stock.
- * La cantidad de productos sin stock.
- * El código y precio del producto más caro.



Problema: Controlar si una secuencia de caracteres está formada solamente por letras, dígitos y tiene un máximo de dos guiones bajos.

Secuencia OK:

ah18lda_jf
298vho_iwf_
__dkg

Secuencia not OK:

dj1i34"
@ldkfj10___kej_
ks_ks_kaj1_

Podemos asumir q tenemos la longitud de la secuencia como dato de entrada. Al finalizar se debe informar si la secuencia estuvo OK o not OK; y en caso de que estuviera OK, reportar la cantidad de letras y la cantidad de dígitos q la forman.

CONOCEMOS LA LONGITUD

RECORRIDO NO EXHAUSTIVO, cortamos:

- Si el carácter es distinto de letra, dígito o _
- Si se encuentran más de dos guiones
- Si se terminó la secuencia

Además,

- Contar letras
- Contar dígitos
- Contar guiones



Problema: Dada una oración que finaliza en punto, se desea saber si hay alguna aparición de doble R. La oración esta toda escrita en mayúsculas.

- En la oración:

LA CASA ESTABA VACÍA.

No hay doble R.

- En la oración:

ARRIBA LAS MANOS.

Hay doble R.

NO CONOCEMOS LA LONGITUD
RECORRIDO NO EXAHUSTIVO, cortamos si:

- Encontramos dos Rs seguidas
- Encontramos el punto



Algoritmo RconRguitarra

DE: secuencia de caracteres (oración q finaliza en punto)

DS: hayDobleR (lógico)

Daux: ch (char)

leer(ch1)

hayDobleR $\leftarrow$ falso

Mientras (ch1 $\neq$ '.' y hayDobleR = falso)

leer(ch2)

si (ch1='R' y ch2='R')

entonces hayDobleR $\leftarrow$ verdadero

ch1 $\leftarrow$ ch2

Problema Casi Nula: Se dice que una secuencia es Casi Nula si tiene, entre sus elementos, mayoría de valores 0. Es decir, la cantidad de ceros es mayor a la mitad de la cantidad de elementos de la secuencia. Si la secuencia tiene 7 elementos y hay 4, 5, 6 o 7 que son 0, se dice que la secuencia es Casi Nula. Si la secuencia tiene 4 elementos y 0, 1 o 2 de ellos son 0, la secuencia no es Casi Nula.

Escriba un programa que reciba una secuencia de números positivos y determine si es Casi Nula.

Atención: Hacer dos versiones, una donde conocemos la longitud de la secuencia de antemano, y otra donde el ingreso de elementos finaliza con el terminador -1. Analice con cuidado con qué casos se puede optimizar el código.



TERMINADOR

CUANDO PARAR:

- Al encontrar al terminador

Registrar cuantos elementos hay en la secuencia, además de contar los que son cero.

Conozco la longitud

CUANDO PARAR:

- Si la cantidad de ceros supera la mitad de la longitud, ya puedo decir que la secuencia es Casi Nula.
- Cuando se termina la secuencia

EJERCICIOS RESUELTOS



Problema Número Sin D: Decimos que un número N es SinD si en todo el número N , el dígito D no aparece.

Por ejemplo:

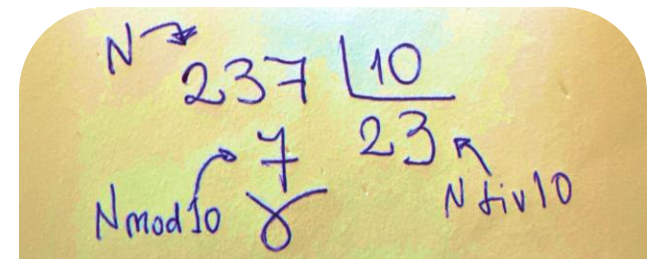
Si N es 23 y D es 5, N es Sin D.

Si N es 118 y D es 9, N es Sin D.

Si N es 54 y D es 4, N no es Sin D. Ya que el dígito 4 aparece en N .

Escriba un algoritmo, y luego el correspondiente programa, que reciba como datos de entrada un número N (natural) y un dígito D (entre 0 y 9), y determine si N es Sin D.

Validar en el programa los datos de entrada, para trabajar con los números cumpliendo con las propiedades pedidas.



Algoritmo NumSinD

DE: N (natural), D (un dígito)

DS: esSinD (lógico)

esSinD $\leftarrow$ verdadero

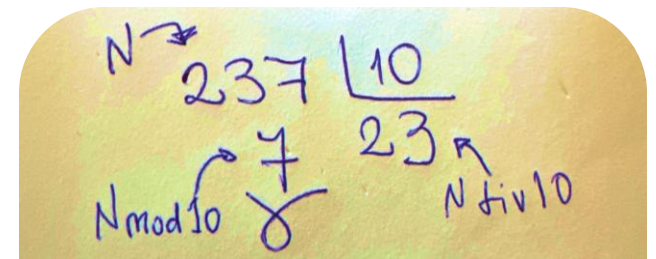
Mientras (N tiene dígitos) y (esSinD es verdadero)
hacer

 Si el último dígito de N es igual a D
 entonces

 esSinD $\leftarrow$ falso

 sino

 N $\leftarrow$ N div 10



Algoritmo NumSinD

DE: N (natural), D (un dígito)

DS: esSinD

Daux: ultD

esSinD $\leftarrow$ verdadero

Mientras (N > 0) y (esSinD = verdadero)

hacer

 ultD $\leftarrow$ N mod 10

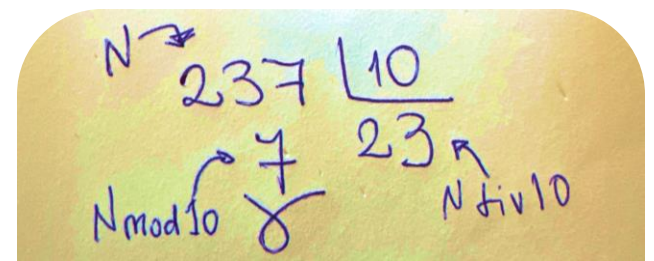
 Si (ultD = D)

 entonces

 esSinD $\leftarrow$ falso

 sino

 N $\leftarrow$ N div 10



```

program NdinD;
var N, D, UltD, N_original: integer;
    esSinD: boolean;

```

```

begin

```

```

    // Ingreso y validación de N natural

```

```

    repeat

```

```

        write(' Ingrese un número natural. ');

```

```

        readln(N);

```

```

        if (N<=0) then writeln(' Mal ingresado.');
```

```

    until (N > 0);

```

```

    // Ingreso y validación de D (dígito)

```

```

    repeat

```

```

        write(' Ingrese un dígito (una cifra entre 0 y 9). ');

```

```

        readln(D);

```

```

        if ((D<0) Or (D>9)) then writeln(' Mal ingresado.');
```

```

    until ((D >= 0) and (D<=9));

```

```

    // Resolución de problema

```

```

    esSinD := true;

```

```

    N_original := N;

```

```

    while ((N>0) and (esSinD)) do

```

```

        // mientras el numero N tenga digitos

```

```

        // y no haya encontrado a D en N

```

```

    begin

```

```

        ultD := N mod 10;

```

```

        if (ultD = D)

```

```

            then esSinD := false

```

```

            else N := N div 10;

```

```

    end;

```

```

    // Reporte de los resultados

```

```

    writeln; writeln;

```

```

    if (esSinD)

```

```

    then writeln(' En el número ', N_original, ' no hay ninguna aparición del dígito ', D)

```

```

    else writeln(' En el número ', N_original, ' hay al menos una aparición del dígito ', D);

```

```

end.

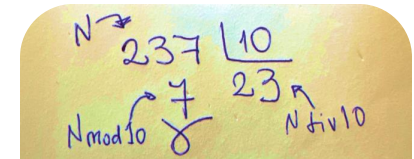
```

```

Ingrese un número natural.    -9
Mal ingresado.
Ingrese un número natural.     0
Mal ingresado.
Ingrese un número natural.    14
Ingrese un dígito (una cifra entre 0 y 9).    55
Mal ingresado.
Ingrese un dígito (una cifra entre 0 y 9).     5

```

En el número 14 no hay ninguna aparición del dígito 5



```

Ingrese un número natural.    5337
Ingrese un dígito (una cifra entre 0 y 9).    3

```

En el número 5337 hay al menos una aparición del dígito 3

Problema Varios Triángulos: Suponga que se ingresa una secuencia con pares de números reales. Cada par corresponde a la base y altura de un triángulo rectángulo. Por ejemplo, para 3 triángulos sería:

5 10 2 3 12 15

Dada esa secuencia, se indica que para el primer triángulo, la base es de 5 cm. y la altura de 10 cm. Para el segundo, la base es de 2 cm. y la altura de 3 cm. Así siguiendo.

Se desea escribir un programa que, dada la cantidad de triángulos (3 en este caso), procese una lista como esta, y muestre por pantalla el área de cada uno de los triángulos.

```
24 lines compiled, 0.1 sec
Ingrese la cantidad de triangulos: 3
Ingrese en secuencia 3 pares de valores reales.
El primer valor del par corresponderá a la base y el segundo, a la altura.
5 6 10 12 4 7
El área del triángulo numero 1 es 15.00
El área del triángulo numero 2 es 60.00
El área del triángulo numero 3 es 14.00
```



Algoritmo variosTriangulos

DE: cantT (natural), *secuencia de pares de números (base, altura)*

DS: *carteles mostrando el área correspondiente a cada par*

Daux: base, altura, área (real), i (natural)

Para i desde 1 hasta CantT hacer

 leer(base, altura)

$\text{área} \leftarrow \text{base} * \text{altura} / 2$

 mostrar(área)





```
1
2 Program variosTRIANGULOS;
3 Var base, altura, area: real;
4     i, cantT: integer;
5 Begin
6
7 repeat
8     Write('Ingrese la cantidad de triangulos: ');
9     read(CantT)
10 until (CantT >= 0);
11
12 writeln('Ingrese en secuencia ', CantT, ' pares de valores reales.');
```

```
24 lines compiled, 0.1 sec
Ingrese la cantidad de triangulos: 3
Ingrese en secuencia 3 pares de valores reales.
El primer valor del par corresponderá a la base y el segundo, a la altura.
5 6 10 12 4 7
El área del triángulo numero 1 es 15.00
El área del triángulo numero 2 es 60.00
El área del triángulo numero 3 es 14.00
```

Además, queremos mostrar al final
el promedio de todas las áreas.

Problema Varios Triángulos 2: Suponga que se ingresa una secuencia con pares de números reales. Cada par corresponde a la base y altura de un triángulo rectángulo. Por ejemplo, para 3 triángulos sería:

5 10 2 3 12 15

Dada esa secuencia, se indica que para el primer triángulo, la base es de 5 cm. y la altura de 10 cm. Para el segundo, la base es de 2 cm. y la altura de 3 cm. Así siguiendo.

Se desea escribir un programa que, dada la cantidad de triángulos (3 en este caso), procese una lista como esta, y muestre por pantalla el área de cada uno de los triángulos. Además, se quiere mostrar cuál de los triángulos fue el más grande.



```

1
2 Program variosTRIANGULOS2;
3 Var base, altura, area, maxArea: real;
4     i, cantT, elMayor: integer;
5 Begin
6
7 repeat
8     Write('Ingrese la cantidad de triangulos: ');
9     read(CantT)
10 until (CantT >= 0);
11
12 maxArea := 0;
13
14 writeln('Ingrese en secuencia ', CantT, ' pares de valores reales. ');
15 writeln('El primer valor del par corresponderá a la base y el segundo, a la altura. ');
16 for i:= 1 to CantT do
17     Begin
18         read(base, altura);
19         area:=base*altura/2;
20         writeln('El área del triángulo numero ', i, ' es ', area:2:2);
21         if (area > maxArea)
22             then
23                 Begin
24                     maxArea := area;
25                     elMayor := i;
26                 End
27             End;
28 writeln('El mayor triángulo fue el triángulo numero ', elMayor);
29 End.

```

```

Ingrese la cantidad de triangulos: 2
Ingrese en secuencia 2 pares de valores reales.
El primer valor del par corresponderá a la base y el segundo, a la altura.
4 7 8 8
El área del triángulo numero 1 es 14.00
El área del triángulo numero 2 es 32.00
El mayor triángulo fue el triángulo numero 2

```



Problema Varios Triángulos 3: Suponga que se ingresa una secuencia con pares de números reales. Cada par corresponde a la base y altura de un triángulo rectángulo. Se considera que termina la secuencia cuando se encuentran dos ceros. Por ejemplo, para 3 triángulos sería:

5 10 2 3 12 15 0 0

Dada esa secuencia, se indica que para el primer triángulo, la base es de 5 cm. y la altura de 10 cm. Para el segundo, la base es de 2 cm. y la altura de 3 cm. Así siguiendo.

Se desea escribir un programa que procese una lista como esta, y muestre por pantalla el área del triángulo más pequeño.



```

1
2 Program variosTRIANGULOS3;
3
4 Var base, altura, area, minAREA: real;
5
6 Begin
7
8   minAREA := MAXINT;
9
10  writeln('Ingresa, en secuencia, pares de valores reales, finalizando con el par 0 0.');
```

11 writeln('El primer valor del par corresponderá a la base y el segundo, a la altura.');

```

12  repeat
13      read(base, altura);
14      if (base <> 0) and (altura<>0)
15      then
16          begin
17              area:=base*altura/2;
18              if (area < minAREA)
19              then
20                  minAREA:=area;
21              end
22  until (base=0) and (altura=0);
23
24  writeln('El área del menor triángulo fue de ', minAREA:2:3, 'cm2.');
```

25 End.

```

Ingresa, en secuencia, pares de valores reales, finalizando con el par 0 0.
El primer valor del par corresponderá a la base y el segundo, a la altura.
3 5 76 8 20 10 0 0
El área del menor triángulo fue de 7.500cm2.
```

